



创新创业实践

Project6

说明文档

郎泓森

202200460010

一、数学推导表示

协议数学表示

设两方为 P_1 和 P_2 :

P_1 持有标识集合 $V = \{v_i\}_{i=1}^{m_1}$

P_2 持有键值对集合 $W = \{(w_j, t_j)\}_{j=1}^{m_2}$

密码学基础

1. 椭圆曲线参数:

素数域 F_p , 其中 p 为大素数

椭圆曲线方程: $y^2 = x^3 + ax + b \bmod p$

基点 G , 阶为 n (代码中使用 NIST P-256 曲线)

2. 哈希函数:

$H: U \rightarrow E(F_p)$ (标识到椭圆曲线点的映射)

实现: `hash_to_point()` 使用 HKDF-SHA256 和 Tonelli-Shanks 算法

3. 加法同态加密:

Paillier 加密系统: $(AGen, AEnc, ADec)$

满足 $AEnc(m1) \oplus AEnc(m2) = AEnc(m1 + m2)$

协议流程推导

Setup:

1. $P1$ 随机选择 $k_1 \leftarrow [1, n - 1]$
2. $P2$ 生成 Paillier 密钥对 $(pk, sk) \leftarrow AGen(\lambda)$ 并发送 pk 给 P_1

Round 1 ($P1 \rightarrow P2$):

$$\forall v_i \in V: M_i = [k_1]H(v_i)$$

M_i 是椭圆曲线点 (代码中序列化为 x 坐标)

发送随机排列的 $\{M_i\}_{i=1}^{m_1}$

Round 2 ($P2 \rightarrow P1$):

1. 计算双掩码:

$$\forall M_i: Z_i = [k_2]M_i = [k_1 k_2]H(v_i)$$

发送随机排列的 $\{Z_i\}_{i=1}^{m_1}$

2. 处理自身集合.

Round 3 ($P1 \rightarrow P2$):

1. 计算双掩码:

$$\forall A_j: A'_j = [k_1]A_j = [k_1 k_2]H(w_j)$$

2. 求交集:

$$J = \{j \mid A'_j \in \{Z_i\}_{i=1}^{m_1}\}$$

交集基数 $C = |J|$ (双方可知)

3. 同态求和:

$$S_{enc} = \bigoplus_{j \in J} B_j = AEnc(\sum_{j \in J} t_j)$$

发送 $ARefresh(S_{enc})$ (刷新随机性)

Output ($P2$):

- 解密: $S = ADec(sk, S_{enc})$
- 输出: S (关联值之和)

二、关键算法实现

1. 椭圆曲线运算

```

def _point_double(self, x, y):
    """点加倍运算:  $P \rightarrow [2]P$ """
     $\lambda = ((3*x^2 + a) * \text{mod\_inv}(2y, p)) \% p$ 
     $x_3 = (\lambda^2 - 2x) \% p$ 
     $y_3 = (\lambda(x - x_3) - y) \% p$ 
    return x3, y3

def _scalar_multiply(self, x, y, k):
    """标量乘法:  $Q = [k]P$  (double-and-add)"""
    Q = (0, 0) # 无穷远点
    P = (x, y)
    while k:
        if k & 1:
            Q = point_add(Q, P)
            P = point_double(P)
            k >>= 1
    return Q

```

2. 哈希到曲线

```

def _hash_to_point(self, identifier):
    """将标识符映射到椭圆曲线点"""
    # HKDF 扩展确保均匀分布
    hkdf = HKDF(SHA256(), output_length=32)
    key_bytes = hkdf.derive(identifier.encode())

    # 循环直到找到曲线上的点
    x = int_from_bytes(key_bytes) % p
    while True:
         $y^2 = (x^3 + a*x + b) \% p$ 
        if is_quadratic_residue(y2, p):
            y = modular_sqrt(y2, p) # Tonelli-Shanks
            return (x, y)
        x = sha256(x).digest() # 下一轮尝试

```

3. 协议流程

```

# P1 Round 1
def p1_round1(self, v_set, k1):
    hashed_exponents = []
    for v in v_set:
        x, y = self._hash_to_point(v)
        x_res, _ = self._scalar_multiply(x, y, k1) # [k1]H(v)
        hashed_exponents.append(x_res.to_bytes(32, 'big'))
    return shuffled(hashed_exponents)

```

```

# P2 Round 2
def p2_round2(self, received, w_pairs, k2, pk):
    # 计算双掩码 Z
    Z = [self._scalar_multiply(int_from_bytes(rb), k2).to_bytes()
          for rb in received]

    # 处理自身集合
    encrypted_pairs = []
    for w, t in w_pairs:
        x, y = self._hash_to_point(w)
        x_res, _ = self._scalar_multiply(x, y, k2) #  $[k_2]H(w)$ 
        enc_t = pk.encrypt(t) #  $AEnc(t_j)$ 
        encrypted_pairs.append((x_res.to_bytes(32, 'big'), enc_t))

    return shuffled(Z), shuffled(encrypted_pairs)

```

4.运行结果

曲线参数：

模数 (p): 0xffffffff0000000100...

阶数 (order): 0xffffffff00000000ff...

生成Paillier密钥对...

生成随机指数...

k1: 0xe6945515325d8294f7...

k2: 0xcfd0c8885849ce375...

开始协议执行...

P1数据: ['user1', 'user2', 'user3']

P2数据: [('user1', 100), ('user2', 200), ('user3', 300), ('user4', 400)]

期望交集和: 600 (user1+user2+user3)

=== Round 1: P1 → P2 ===

P1发送了 3 个点的列表

=== Round 2: P2 → P1 ===

P2发送了Z (3点) 和 4 个加密对

=== Round 3: P1 → P2 ===

检测到 3 个交集元素

P1检测到交集并计算加密和

P2解密结果...

=== 结果 ===

计算交集和: 600

期望交集和: 600

✅ 测试成功! 协议正确执行

根据运行结果显示，代码成功生成了 paillier 密钥对并进行了 3 轮交互，计算交集和与期望和相同，协议正确执行。

数学安全性分析

1. 隐私保护原理

标识隐藏：攻击者无法从 $[k]H(v)$ 反推 v (ECDLP 困难问题)

关联值隐藏：Paillier 加密的语义安全性保护 t_j

双掩码机制： $H(v_i)^{k_1 k_2}$ 在 DDH 假设下与随机点不可区分

2. 正确性证明

当 $v_i = w_j$ 时：

$$\begin{aligned} [k_1]H(v_i) &= M_i \\ [k_2]M_i &= [k_1 k_2]H(v_i) = Z_i \\ [k_1]A_j &= [k_1 k_2]H(w_j) = [k_1 k_2]H(v_i) \end{aligned}$$

故 $A'_j \in \{Z_i\}$ ，交集检测正确。

3. 复杂度优化

点运算：double-and-add 算法复杂度 $O(\log k)$

批量处理：Paillier 支持同态加法 $\oplus_{j \in JB} j$

通信优化：仅传输椭圆曲线点 x 坐标 (32 字节)